

PROGRAMMERINGSBOK FÖR INGENJÖR 1997

Föreläsning 1

Ändrad
24 Sep
13:57

-- några rudimentära exempel

Utan vidare dröjsmål, låt oss börja med ett exempel. Detta är ett C-program, ett program i språket C:

```
1  #include <stdio.h>
2
3  void main(){
4
5      int i, j;
6
7      i = 5;
8      j = 4 * (i + 20);
9
10     printf("Summan av %d och %d är %d.\n", i, j, i + j);
11 }
```

Ett program är en följd instruktioner till en dator. Datorn följer instruktionerna i tur och ordning uppifrån och ner.

Låt oss titta på programmet rad för rad (raderna är numrerade bara för att vi ska kunna prata om dem. I det riktiga programmet finns inga radnummer).

Tilldelning, variabel, deklaration, aritmetik

Vi börjar med att hoppa ner en bit i programmet. Där det börjar hända nåt:

```
7  i = 5;
```

En variabel som heter `i` får värdet `5`. En *variabel* är en minnesplats. Variabeln "minns" exakt ett värde i taget.

Att ge värden åt variabler, det är det som programmering går ut på -- om man spetsar till det lite. Att räkna ut ett värde, och att sedan ge en variabel detta värde.

Rad `7` kallas *tilldelningssats*. Variabeln ska stå till vänster, och värdet till höger.

Variabeln `i` kan bara minnas heltal. Detta bestäms på rad `5`:

```
5  int i, j;
```

Här anges att variablerna `i` och `j` är av typen `int`. Det vill säga, de minns *heltal* (eng. *integer*). Raden kallas *deklarationssats*. Det heter att variablerna *deklarerar*s.

På rad `8` får `j` ett värde i en tilldelningssats:

```
8  j = 4 * (i + 20);
```

Det som står till höger får -- som synes -- vara ett

aritmetiskt uttryck. Reglerna för aritmetiska uttryck är desamma som inom matematiken. Tecknet `*` är multiplikation. Tecknet kan inte utelämnas -- i matematiken skriver vi ju $4(i+20)$.

Utskrift: `printf`

Slutligen ska datorn skriva ut det den kommit fram till:

```
10 printf("Summan av %d och %d är %d.\n", i, j, i + j);
```

Instruktionen heter `printf`. Det betyder "skriv!". Texten som ska skrivas är det som står inom parenteserna.

Det här är vad som faktiskt skrivs ut när programmet körs:

```
1 Summan av 5 och 100 är 105.
```

Jämför rad 10 med det som skrivs ut! De tre `%d` byts i tur och ordning ut mot de tre värden som räknas upp med kommatecken emellan. Variablerna `i` och `j` samt deras summa skrivs ut.

Det avslutande `\n` i citatet betyder *ny rad*, (eng. *newline*).

Semikolon

Många rader i exemplet avslutas med semikolon, `;`. Semikolonerna talar om för datorn var *satser* börjar och slutar. Hittills har vi bland annat stött på en *deklarationssats* och *tilldelningssats*.

Ibland kan en sats sträcka sig över flera rader. Ibland kan man ha flera satser på en rad. Därför behövs semikolonerna -- så att datorn vet var satsen slutar. Datorn bryr sig inte om var rader slutar. För datorns del kunde nästan lika gärna hela programmet stå på en rad. Följande layout är fullt tillåten:

```
#include <stdio.h>
void main(){ int i, j; i = 5; j = 4 * (i + 20);
printf("Summan av %d och %d är %d.\n",
i, j, i + j); }
```

Datorn bryr sig bara om var semikolonerna sitter (med ett par undantag). Däremot bryr sig naturligtvis Thomas och Björn, de som ska rätta era laborationer.

`include`

Rad 1 är en "include"-sats. Den säger att programmet kommer att kommunicera med en användare:

```
1 #include <stdio.h>
```

Kommunikationen ska ske med hjälp av tangentbord och

textskärm. Ingen mus eller grafik, med knappar och annat lyx. Användaren ska skriva text, och datorn ska svara med text på bildskärmen. Det var det som var standard på den tiden språket C skapades. Och `stdio` betyder mycket riktigt *standard input/output*.

Rad 1 byts automatiskt ut mot en fil som heter `stdio.h`. Där står vad `printf` betyder. Om vi tar bort rad 1, kommer datorn att fråga vad `printf` betyder.

Tecknet `#` i "include"-satsen *måste* stå allra först på en rad. Annars "ser" inte datorn satsen. Detta är ett undantag från regeln att datorn bara bryr sig om var semikolonen sitter.

Nästa lektion

